

ELEG 55903: Programming for Power Electronics

HW8

Tyler Cash

ELEG 4593

HW8

13 April 2026

Abstract:

In this homework assignment, we expand previous assignments and incorporate an LED Control module into the VHDL project. The LED control module consists of 4 PWM submodules as well as a Bus Interface to read control parameters from the SPRAM. The Four LEDs will be controlled by using the provided labView interface via the Serial Port.

Table of Contents:

Abstract:.....	2
List of Figures:.....	3
Overview:.....	3
Modules:.....	3
PWMs:.....	3
Testbench:.....	4
Conclusion:	4
Appendix A (Bus Interface Top):	8
Appendix B (LED_Cntrl):.....	9

List of Figures:

FIGURE 1. SERIAL /REG/PWM INTERFACE WAVEFORM.....	5
FIGURE 2. 100% ALL LED INTENSITY, 100% BLINK FREQ, 100% ON TIME, 100%.....	6
FIGURE 3. 100% LED INTENSITY.....	7
FIGURE 4. 5% LED INTENSITY.....	8
FIGURE 5. 1% LED INTENSITY.....	9

Overview:

This project was an expansion of our previous homework assignments and midterm. We added an LED Control module, with 4 PWM submodules to our previous bus interface/uart serial interface. This allows us to control the blink frequency, on time, and intensity of the LEDs on our MachXO3D breakout board, by writing commands using the uart serial interface.

Modules:

For this assignment, there we build mostly on top of our previous work, only needing to make some changes to the Bus Interface Top Module and then creating the LED Control module. The logic for the PWMs, as well as the control of the LED lights all lives in the LED control Module.

PWMs:

This project made use of 4 PWM submodules to control the LEDs. The PWM, Pulse Width Modulation, controls the amount of power to the LEDs based on our PWM period, which is based on a counter we are iterating per clock cycle. The LEDs are turning off based on the period we have defined, but the switching is so fast that the human eye cannot perceive the LEDs turning off, but instead see it as more intense, or less intense light. Figure 2, 3, 4, and 5 show the board running with different intensity of the LEDs. Figure 2 shows the board with 100% blink rate, 100% on time, and all LEDs set to 100% intensity. Figure 3 Shows the board with LED lights set to 75% blink frequency, 50% on time, and then with LED1 with 12.5% intensity, LED2 to 25% intensity, LED3 set to 50% intensity, and LED4 set to 100% intensity, and you can see the difference between each LED quite clearly in the image. Figure 4 has the LEDs all set to 5% intensity, and then Figure 5 has all LEDs set to 1% intensity.

Testbench:

The testbench is like the Midterm testbench, beginning with a reset and then writing commands using the UART serial interface. The resulting waveform is located in Figure 1 and shows the 4 PWMs operating at different duty cycles, with LED1 operating at 12.5%, LED2 operating at 25%, LED3 operating at 50%, and LED4 operating at 100%

duty cycle. The resulting waveform can be seen in Figure 1. The other testing results, already mentioned as the other figures, were demonstrated writing specific commands to the breakout board. The start delimiter for the commands is 0x7E, followed by packet length 0x12, followed by OPID 0x0A, then register count 0x07, then the Register Data, and finally a checksum. The checksum is calculated by summing all the bytes between the packet length and the checksum and then subtracting 0xFF.

Conclusion:

In conclusion, we properly incorporated an LED Control Module, with multiple PWM submodules, into our VHDL project. This allows us to control the LEDs on the breakout board by sending packets of data using the serial port.

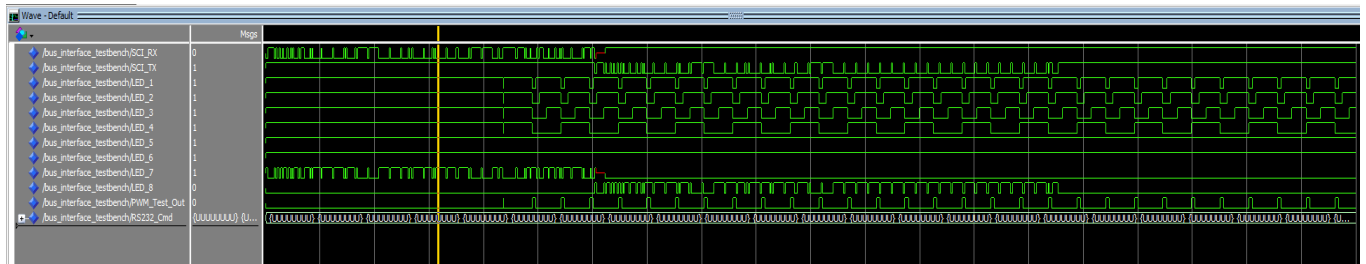


Figure 1. Serial /Reg/PWM Interface Waveform

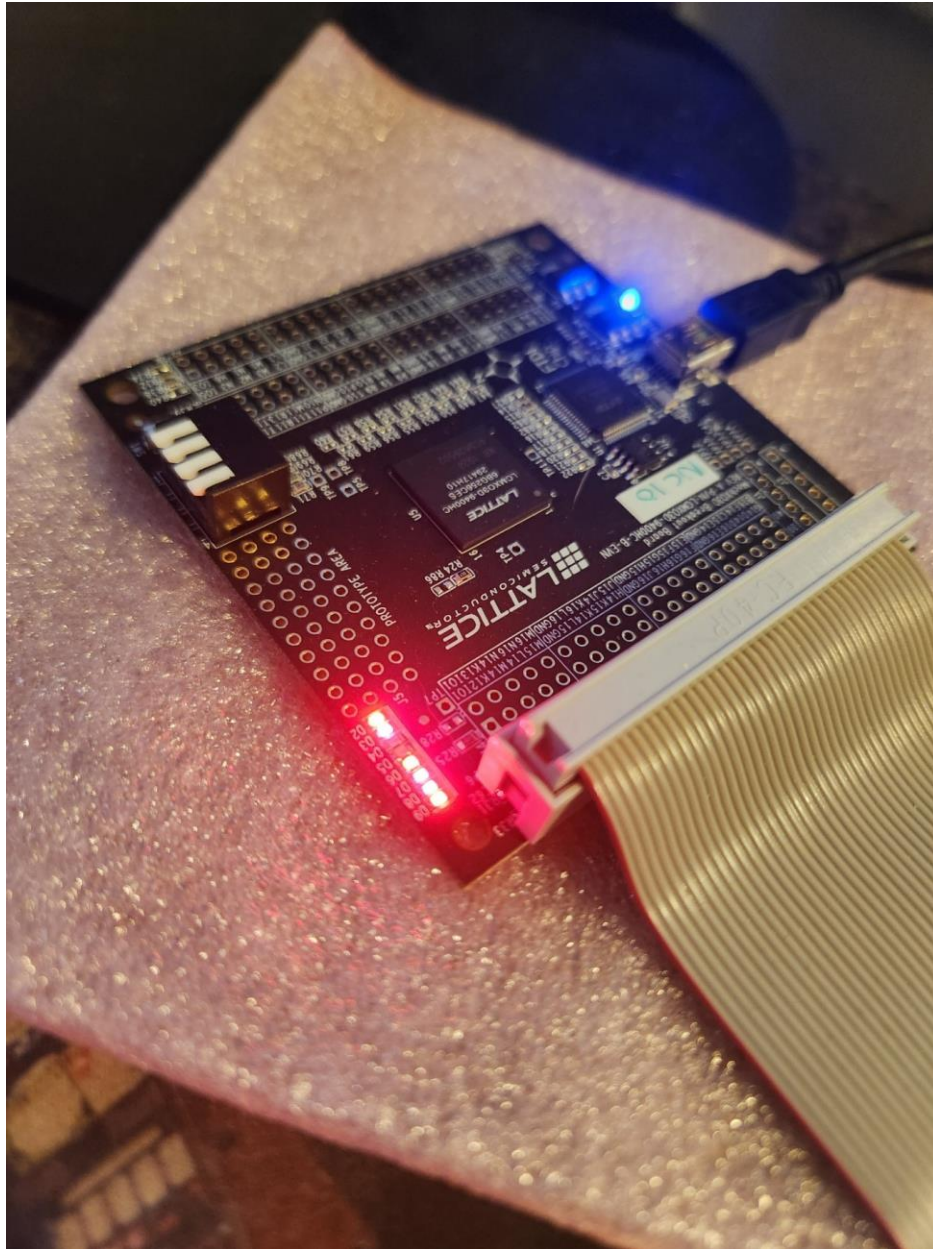


Figure 2. 100% All LED Intensity, 100% Blink Freq, 100% On Time, 100%

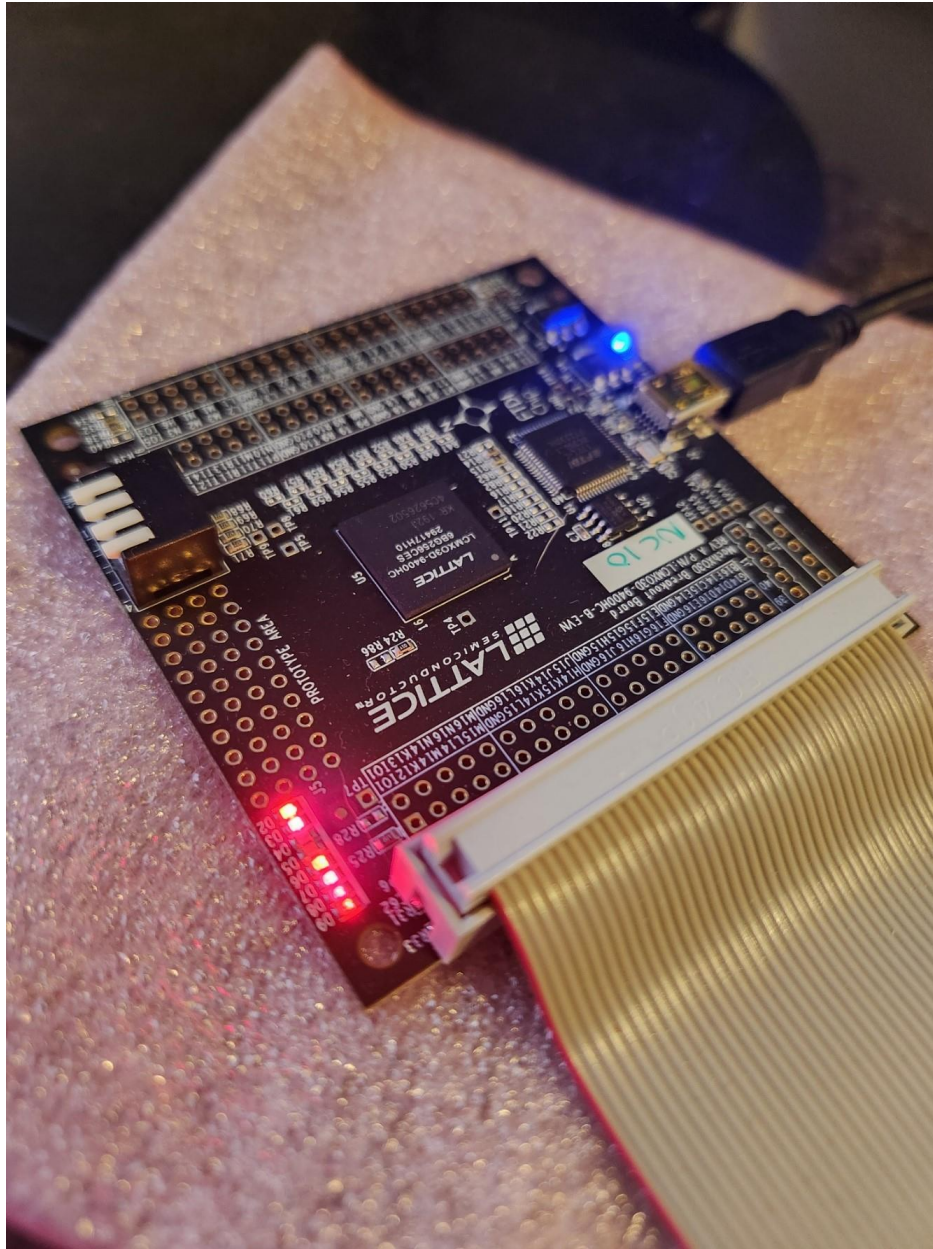


Figure 3. 100% LED Intensity

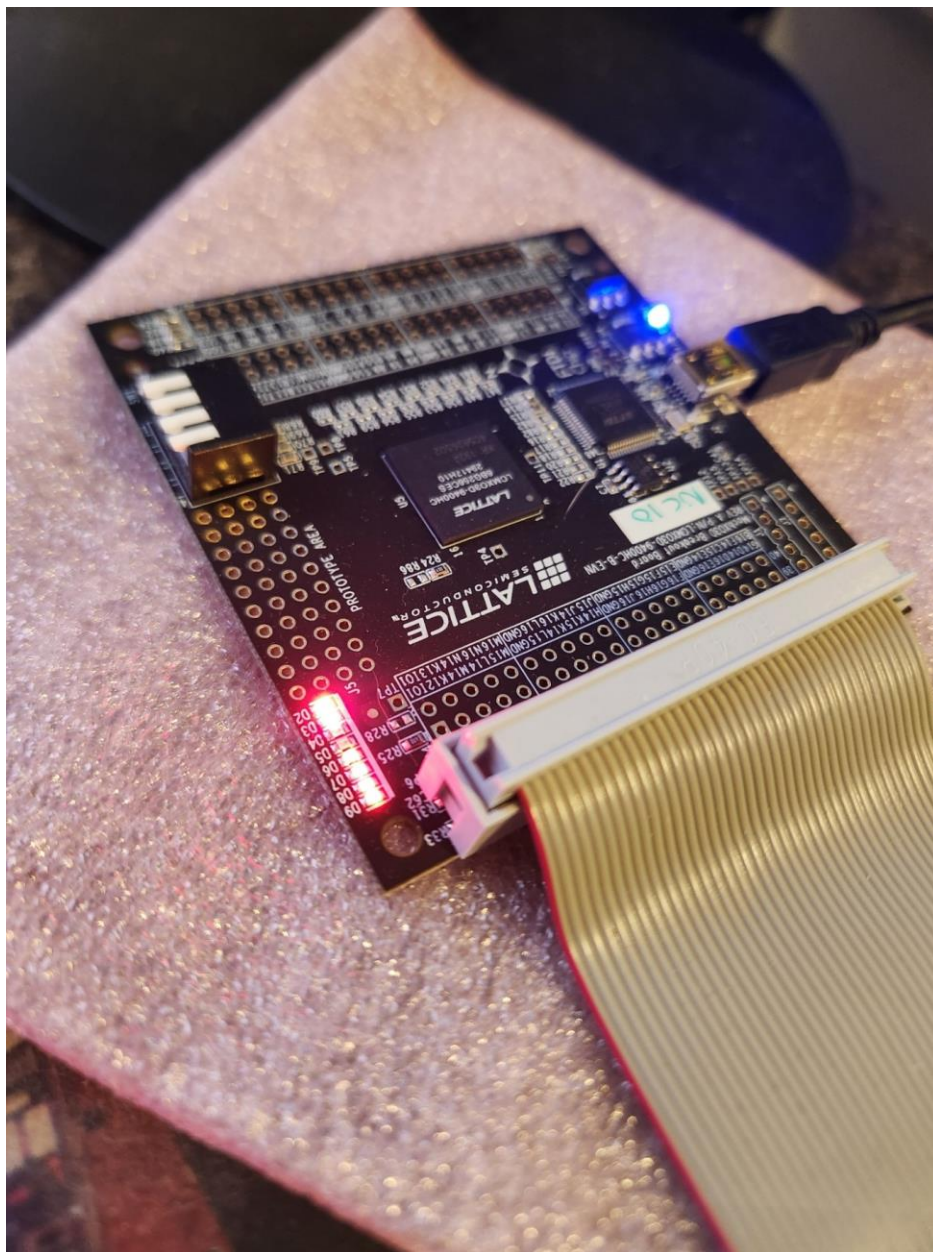


Figure 4. 5% LED Intensity

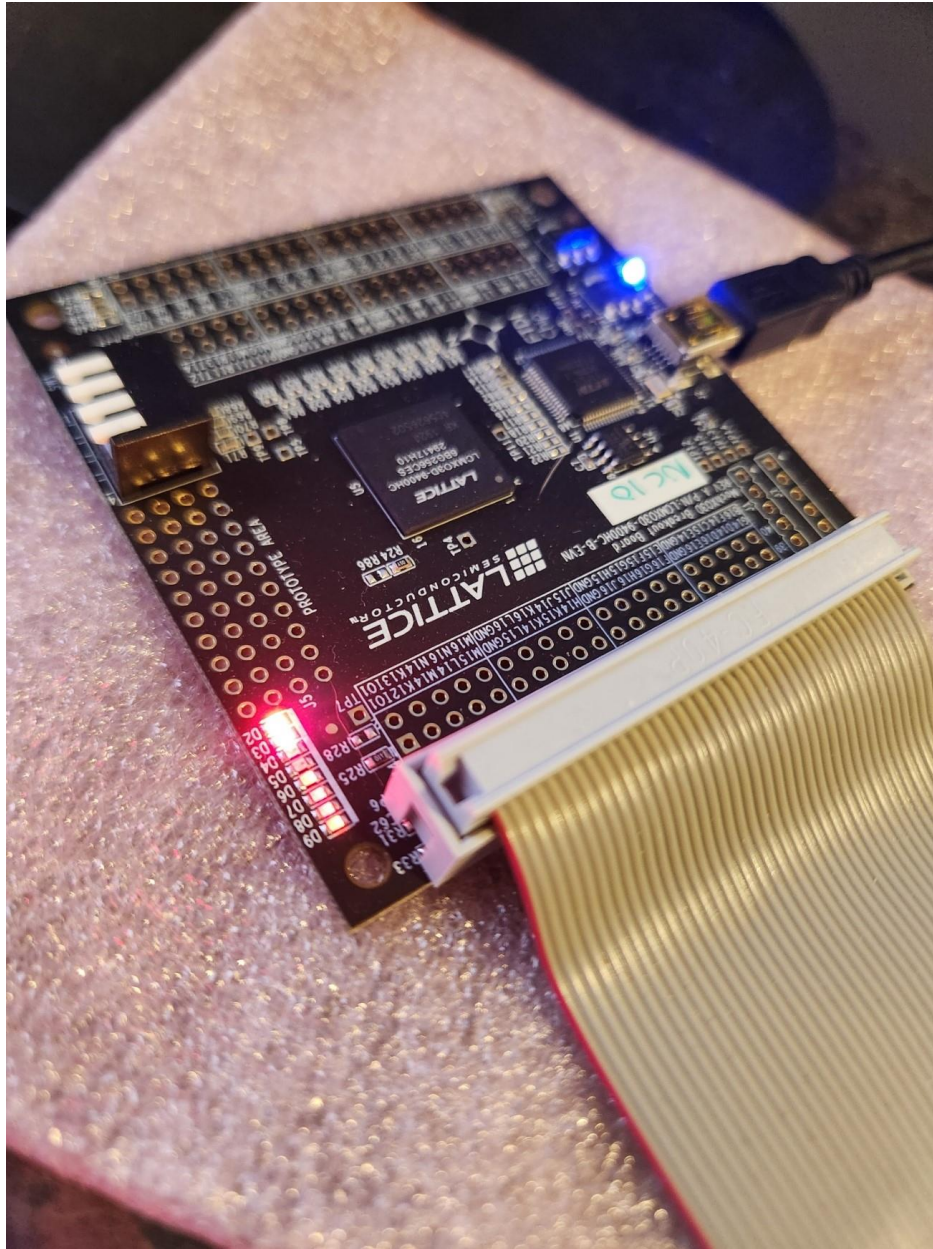


Figure 5. 1% LED Intensity

Appendix A (Bus Interface Top):

```
-- Company: University of Arkansas (NCREPT) -- Engineer: Tyler Cash -- --
Create Date: 25March2026 -- Last Updated: 7AprilMar2026 -- Design Name: Bus
Interface w/ UART -- Module Name: BusInterface_UART - Behavioral -- Project
Name: BusInterface_UART -- Target Devices: LCMX03D-9400HC-6BG256C
(MachX03D_BreakoutBrd) -- Tool versions: Lattice Diamond_x64 Build
3.12.0.240.2 -- Description: -- This Project was developed to demonstrate a
system-level design which incorporates shared registers and communications. -
- You may use various Serial Interface programs to communicate with this
including the old version of X-CTU (Version 5.2.8.6), -- Termite,
HypterTerminal, or the Custom LabVIEW Interface created for this project
(LabVIEW-CommEx_Bus_Interface_PWM_v1.0a_16Apr2021). -- A key to remember is
that you will need to create packets in Hex-Mode not the standard ASCII-Mode.
-- A Testbench for this project was created and demonstrates base
functionality. -- ---- PinOut: -- Signal CPLD_Pin Description -- SCI_TX C11
Serial-TX (Output) [USB Serial Port on Breakout Board] -- SCI_RX A11 Serial-
RX (Input) [USB Serial Port on Breakout Board] -- ---- Register and Memory
Map Information: -- This section describes the Memory Map used in this
project. -- This design contains a SPRAM Module which is 16 bits wide and
1024 entries deep. -- Register addresses are from X"0000" to X"03FF". -- All
registers are 16-bits wide. -- The SPRAM Module is located in the Bus_Master
portion of the code. -- This RAM Module may be accessed externally using
Serial Port interface. -- Reserved for future use. -- X"0300" - X"03FF" --
Register Map is found as constants in Bus_Interface_Common and shared with
all submodules of this program. -- ---- Serial Settings -- Baud: 9600 -- Flow
Control: None -- Data Bits: 8 -- Parity: None -- Stop Bits: 1 -- -- -- ----
Serial Communications: -- In the project we will construct packets to
facilitate serial communication between the device and a computer. -- Using
packets we can implement operational commands and check sums which allow for
easily implementing a user GUI. -- You may use various Serial Interface
programs to communicate with this including the old version of X-CTU (Version
5.2.8.6), -- Termite, HypterTerminal, or the Custom LabVIEW Interface created
for this project (LabVIEW-CommEx_Bus_Interface_PWM_v1.0a_16Apr2021) -- A key
to remember is that you will need to create packets in Hex-Mode not the
standard ASCII-Mode. -- All packets will begin with a Start Delimiter (0x7E)
and end with a CheckSum. -- The CheckSum is calculated by adding up all the
bytes of the packet between the packet length and the checksum then --
subtracting the sum from 0xFF. -- We will implement two types of packets for
our use. -- Register Write Command Packet (Used to update registers internal
to the CPLD) -- Register Read Command Packet (Used to read registers internal
to the CPLD) -- ---Serial Write Packet (OP_ID = 0x0A) -- The Register Write
Command Packet is used to update registers internal to the CPLD. -- The
```



```

updated to allow for Read and Write Command Verification -- ---- Revision
1.1a - -- Updated to use Protocols based on Zigbee Implementation -- --
Revision 1.0b - -- Updated to use UCB instead of Evaluation Board -- --
Revision 0.01 - -- File Created; Basic\Classical Operation Implemented -- --
-- Additional Comments: -- --10

```

```

Library IEEE; use IEEE.std_logic_1164.all; use ieee.numeric_std.all; use
IEEE.std_logic_arith.all;

```

```

library machxo3d; use machxo3d.all;

```

```

library work; use work.Bus_Interface_Common.all;

```

```

entity Bus_Interface_Top is Port ( ---- RS232 Communication SCI_RX : in
STD_LOGIC; SCI_TX : inout STD_LOGIC;

```

```

    LED_1 : out STD_LOGIC;
    LED_2 : out STD_LOGIC;
    LED_3 : out STD_LOGIC;
    LED_4 : out STD_LOGIC;
    LED_5 : out STD_LOGIC;
    LED_6 : out STD_LOGIC;
    LED_7 : out STD_LOGIC;
    LED_8 : out STD_LOGIC;
    PWM_Test_Out : out STD_LOGIC
);

```

```

end Bus_Interface_Top;

```

```

architecture Behavioral of Bus_Interface_Top is Component OSCJ GENERIC
(NOM_FREQ: string := "8.31"); PORT ( STDBY : IN std_logic; OSC : OUT
std_logic; SEDSTDBY : OUT std_logic ); END COMPONENT;

```

```

-- Declare PLL Component PLLSerial PORT ( ClkI: in std_logic; ClkOP: out
std_logic; Lock: out std_logic ); END COMPONENT;

```

```

-- Declare Bus Master COMPONENT Bus_Master PORT ( clk : IN std_logic; rst :
IN std_logic; Data : INOUT STD_LOGIC_VECTOR(15 downto 0); Addr : IN
STD_LOGIC_VECTOR(15 downto 0); Xrqst : IN std_logic; XDat : OUT std_logic;
YDat : IN std_logic; BusRqst : IN std_logic_vector(9 downto 0); BusCtrl : OUT
std_logic_vector(9 downto 0) ); END Component;

```

```

COMPONENT RS232_Usr_Int

```

```

    generic(
        Baud      : integer;
        clk_in     : integer
    );

```

```

Port (
    clk : in STD_LOGIC;
    rst : in STD_LOGIC;
    rs232_rcv : in STD_LOGIC;

```

```

rs232_xmt :    out STD_LOGIC;
Data      :    INOUT std_logic_vector(15 downto 0);
Addr      :    OUT    std_logic_vector(15 downto 0);
Xrqst     :    OUT    std_logic;
XDat      :    IN     std_logic;
YDat      :    OUT    std_logic;
BusRqst   :    OUT    std_logic;
BusCtrl   :    IN     std_logic
);
end    Component;

```

```

-- Declare LED_Ctrl
COMPONENT LED_Ctrl is
Port (
    clk      :    in STD_LOGIC;
    rst      :    in STD_LOGIC;
    Data     :    INOUT std_logic_vector(15 downto 0);
    Addr     :    OUT    std_logic_vector(15 downto 0);
    Xrqst    :    OUT    std_logic;
    XDat     :    IN     std_logic;
    YDat     :    OUT    std_logic;
    BusRqst  :    OUT    std_logic;
    BusCtrl  :    IN     std_logic;
    LED_En   :    out std_logic;
    LED1_Out :    out STD_LOGIC;
    LED2_Out :    out STD_LOGIC;
    LED3_Out :    out STD_LOGIC;
    LED4_Out :    out STD_LOGIC;
    LED5_Out :    out STD_LOGIC;
    LED6_Out :    out STD_LOGIC;
    LED7_Out :    out STD_LOGIC;
    LED8_Out :    out STD_LOGIC
);
END COMPONENT;

```

```

-- Declare STD_Counter Component
component STD_Counter is
GENERIC
(
    Width : integer--width of Counters
);
Port (INC, rst, clk: in std_logic;

```

```

        Count: out std_logic_vector(Width -1 downto 0)
    );
end component;

--signals
-- Declare Signals for Bus Interface
signal Bus_Int1_WE, Bus_Int1_RE, Bus_Int1_Busy: STD_LOGIC:='0';
signal Bus_Int1_DataIn, Bus_Int1_DataOut, Bus_Int1_AddrIn:
STD_LOGIC_VECTOR(15 downto 0):= (others => '0');

-- Inputs
signal Addr : std_logic_vector(15 downto 0):= (others => '0');
signal Xrqst : std_logic := '0';
signal YDat : std_logic := '0';
signal BusRqst : STD_LOGIC_Vector(9 downto 0):= (others => '0');
signal Data : STD_LOGIC_Vector(15 downto 0):= (others => '0');
signal XDat : STD_LOGIC := '0';
signal BusCtrl : STD_LOGIC_Vector(9 downto 0):= (others => '0');

-- internal clock
signal OSC_Stdbby, OSC_Out, OSC_SEDSTDBY, clk: std_logic := '0';

-- Reset
signal PLL_Lock, System_rst: std_logic := '0';
signal Reset_Cnt_INC, Reset_Cnt_rst: std_logic := '0';
signal Reset_Cnt_out: STD_LOGIC_VECTOR(7 downto 0):= (others => '0');

--Misc
signal LED_En : std_logic := '0';
-- For inverting LED Outputs
signal LED_1n, LED_2n, LED_3n, LED_4n, LED_5n, LED_6n, LED_7n, LED_8n:
STD_LOGIC:= '0';

begin
-- Instantiate Internal Oscillator
Int_OSC: OSCJ Port MAP (
    STDBY => OSC_Stdbby,
    OSC => OSC_Out,
    SEDSTDBY => OSC_SEDSTDBY
);

--Instantiate PLL
PLL_1: PLLSerial PORT MAP (

```



```

        ClkI => OSC_Out,
        ClkOP => clk,
        Lock => PLL_Lock
    );

-- Instantaite Bus_Master
BM: Bus_Master Port Map (
    clk => clk,
    rst => System_rst,
    Data => Data,
    Addr => Addr,
    Xrqst => Xrqst,
    XDat => XDat,
    YDat => YDat,
    BusRqst => BusRqst,
    BusCtrl => BusCtrl
);

-- Instantaite RS232_Usr_Int
RS232_Usr: RS232_Usr_Int
Generic MAP
(
    Baud => 9600,
    Clk_in => Clk_Freq -- Input clk
)
PORT MAP (
    clk => clk,
    rst => System_rst,
    rs232_rcv => SCI_RX,
    rs232_xmt => SCI_TX,
    Data => Data,
    Addr => Addr,
    Xrqst => Xrqst,
    XDat => XDat,
    YDat => YDat,
    BusRqst => BusRqst(1),
    BusCtrl => BusCtrl(1)
);

LED_Ctrl1: LED_Ctrl1 PORT MAP (
    clk => clk,
    rst => System_rst,
    Data => Data,

```

```

    Addr => Addr,
    Xrqst => Xrqst,
    XDat => XDat,
    YDat => YDat,
    BusRqst => BusRqst(0),
    BusCtrl => BusCtrl(0),
    LED_En => LED_En,
    LED1_Out => LED_1n,
    LED2_Out => LED_2n,
    LED3_Out => LED_3n,
    LED4_Out => LED_4n,
    LED5_Out => LED_5n,
    LED6_Out => LED_6n,
    LED7_Out => LED_7n,
    LED8_Out => LED_8n
);

--Instantiate Reset_Cnt_8
Reset_Cnt: STD_Counter
generic MAP
(
    Width => 8
)
port map(
    clk => OSC_Out,
    rst => Reset_Cnt_rst,
    INC => Reset_Cnt_INC,
    Count => Reset_Cnt_out
);

-- Oscillator
OSC_Stdbby <= '0';

-- Tie unused ports to '0'
BusRqst(9 downto 2) <= (others => '0');

-- Reset Block1
Reset_Blkl: Process
begin
    wait until OSC_Out'event and OSC_Out = '1';
    if(PLL_Lock = '0') then
        Reset_Cnt_rst <= '0'; -- active low
    else

```

```

        Reset_Cnt_rst <= '1';
    end if;
end process;

-- Reset Block
Reset_Blck: process
begin
    wait until OSC_Out'event and OSC_Out = '1';
    if(Reset_Cnt_out < X"7F") then
        System_rst <= '0';
        Reset_Cnt_Inc <= '1';
    else
        System_rst <= '1';
        Reset_Cnt_Inc <= '0';
    end if;
end process;

-- LED Invert due to Active Low Configuration on Dev Board
LED_Invert: process(LED_1n, LED_2n, LED_3n, LED_4n, LED_5n, LED_6n, LED_7n,
LED_8n, SCI_RX, SCI_TX)
begin

    LED_8 <= not(SCI_TX);
    LED_7 <= not(SCI_RX);
    LED_6 <= not(LED_6n);
    LED_5 <= not(LED_5n);
    LED_4 <= not(LED_4n);
    LED_3 <= not(LED_3n);
    LED_2 <= not(LED_2n);
    LED_1 <= not(LED_1n);
    PWM_Test_Out <= LED_1n;

end process;

end Behavioral;

```

Appendix B (LED Cntrl):

-- Company: University of Arkansas (RIOT) -- Engineer: Tyler Cash -- -- Create Date: 4/08/2026 -- Last Updated: 4/10/2026 -- Design Name: Bus_Interface_TestBench -- Module Name: Bus_Interface_TestBench - Behavioral -- Project Name: Bus Interface Example -- Target Devices: LCMXO3D-9400HC-6BG256C (MachXO3D_BreakoutBrd) -- Tool versions: Lattice Diamond_x64 Build 3.11.3.469.0 -- ---- Description: -- This Test Bench first sends a write command to update registers used to control the PWM Modules. -- Next it issues a read command to read from the registers. -- Total Simulation Time for these commands is approximately 75 ms. -- -- The Commands sent and recieved are documented in the comments below: -- -- The packet below writes 7 16-bit registers starting at register 0x0100; Values listed below. -- PWM_Enable => 0x0001 (1=Enable; 0=Disable) -- LED_BlinkFreq => 0xBFFF [75%] (BFFF/FFFF) (%) -- LED_OnTime => 0x6000 [50%] (6000/BFFF) (%) -- LED1_Intensity => 0x2000 [12.5%] (2000/FFFF) (%) -- LED2_Intensity => 0x4000 [25%] (4000/FFFF) (%) -- LED3_Intensity => 0x8000 [50%] (8000/FFFF) (%) -- LED4_Intensity => 0xFFFF [100%] (FFFF/FFFF) (%) -- -- Start Delimiter | Pkt Len | Op ID | Register_Cnt | Start Address | Register Data (16bit x Register_Cnt) | ChkSum -- 0x7E | 0x12 | 0x0A | 0x07 | 0x0100 | 0x0001BFFF6000200040008000FFFF | 0xF0 -- 0x7E120A0701000001BFFF6000200040008000FFFFF0 results in all LEDs being set to the above parameters. -- 0x7E120A0701000001BFFF60000000000000000000CE results in all LED Intensities being set to 0%. -- -- The Register Read Command Packet is used to read registers internal to the CPLD. -- The following example breaks down a read request packet. -- The packet below reads 16 16-bit registers starting at register 0x0100. -- Start Delimiter | Pkt Len | Op ID | Register_Cnt | Start Address | ChkSum -- 0x7E | 0x04 | 0x0F | 0x10 | 0x0100 | 0xDF -- 0x7E040F100100DF -- -- The above command results in a write command being sent from the CPLD which contains data from 16 16-bit registers starting at address 0x0100. -- An example response from the CPLD is shown below: -- 0x7E240A1001000001BFFF6000200040008000FFFF00000000000000000000000000000000E7 -- Start Delimiter | Pkt Len | Op ID | Register_Cnt | Start Address | Register Data (16bit x Register_Cnt) | ChkSum

-- 0x7E | 0x24 | 0x0A | 0x10 | 0x0100
| 0x0001BFFF6000200040008000FFFF00000000000000000000000000000000 | 0xE7

--

-- Revisions:-- ---- Revision 2.0a - -- Updated for MachXO3D and so it could serve as an example for the "Programming for Power Electronics" Class. -- Testbed updated to allow for Read and Write Command Verification -- ---- Revision 1.1b - -- Minor Updates to documentation and PWMs. -- Testbed updated to allow for Read and Write Command Verification -- ---- Revision 1.1a - -- Updated to use Protocols based

on Zigbee Implementation -- -- Revision 1.0b -- -- Updated to use UCB instead of Evaluation Board -- --
Revision 0.01 -- -- File Created; Basic\Classical Operation Implemented -- -- -- Additional Comments: -- --

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL; use IEEE.STD_LOGIC_ARITH.ALL; use  
IEEE.STD_LOGIC_UNSIGNED.ALL; use ieee.numeric_std.all;
```

```
library work; use work.Bus_Interface_Common.all;
```

```
entity LED_Ctrl is Port ( clk : in STD_LOGIC; rst : in STD_LOGIC; Data : INOUT std_logic_vector(15 downto  
0); Addr : OUT std_logic_vector(15 downto 0); Xrqst : OUT std_logic; XDat : IN std_logic; YDat : OUT  
std_logic; BusRqst : OUT std_logic; BusCtrl : IN std_logic; LED_En : out std_logic; LED1_Out : out  
STD_LOGIC; LED2_Out : out STD_LOGIC; LED3_Out : out STD_LOGIC; LED4_Out : out STD_LOGIC;  
LED5_Out : out STD_LOGIC; LED6_Out : out STD_LOGIC; LED7_Out : out STD_LOGIC; LED8_Out : out  
STD_LOGIC ); end LED_Ctrl;
```

```
architecture Behavioral of LED_Ctrl is type state_type is (S0, S1, S2, S3, S4, S5, S6, S7, S8, S9, S10, S11,  
S12, S13, S14, S15, S16, S17); signal CS_Bus, NS_Bus, CS_PWM1, NS_PWM1, CS_PWM2, NS_PWM2,  
CS_PWM3, NS_PWM3, CS_PWM4, NS_PWM4: state_type;
```

```
--declare STD_Counter Component  
component STD_Counter is  
generic  
(  
    Width : integer    --width of counter  
);  
port(INC,rst,clk: in std_logic;  
    Count: out STD_LOGIC_VECTOR(Width-1 downto 0));  
end component;
```

```
--declare Bus Interface  
COMPONENT Bus_Int  
PORT(  
    clk    :  IN std_logic;  
    rst    :  IN std_logic;  
    DataIn  :  IN  std_logic_vector(15 downto 0);  
    DataOut :  OUT std_logic_vector(15 downto 0);  
    AddrIn  :  IN std_logic_vector(15 downto 0);  
    WE     :  IN std_logic;  
    RE     :  IN std_logic;  
    Busy   :  OUT std_logic;  
    Data   :  INOUT  std_logic_vector(15 downto 0);  
    Addr   :  OUT  std_logic_vector(15 downto 0);
```

```

Xrqst : OUT std_logic;
XDat : IN std_logic;
YDat : OUT std_logic;
BusRqst : OUT std_logic;
BusCtrl : IN std_logic
);
END COMPONENT;

```

```

--Signals

```

```

signal PWM_En, PWM_PRD, PWM_PW, PWM1_DC, PWM2_DC, PWM3_DC, PWM4_DC:
STD_LOGIC_VECTOR(15 downto 0):= X"0000";
signal PWM1_En, PWM2_En, PWM3_En, PWM4_En: STD_LOGIC:= '0';

```

```

--Max PWM Values

```

```

constant PWM_Max: std_logic_vector(15 downto 0):= X"FFFF";
constant PWM_Min: std_logic_vector(15 downto 0):= X"0000";

```

```

--Declare Signals for Bus Interface

```

```

signal Bus_Int1_WE, Bus_Int1_RE, Bus_Int1_Busy: STD_LOGIC:= '0';
signal Bus_Int1_DataIn, Bus_Int1_DataOut, Bus_Int1_AddrIn: STD_LOGIC_VECTOR(15 downto 0):=
(others => '0');
signal Bus_Cnt_rst, Bus_Cnt_INC: STD_LOGIC:= '0';
signal Bus_Cnt_Out: STD_LOGIC_VECTOR(15 downto 0):= (others => '0');
signal Delay_Cnt_rst, Delay_Cnt_INC: STD_LOGIC:= '0';
signal Delay_Cnt_Out: STD_LOGIC_VECTOR(7 downto 0):= (others => '0');

```

```

--Signals for Registers

```

```

signal LD_PWM_En, LD_PWM_PRD, LD_PWM_PW, LD_PWM1_DC, LD_PWM2_DC, LD_PWM3_DC,
LD_PWM4_DC: STD_LOGIC:= '0';

```

```

--Signals for PWM Counters

```

```

signal PWM1_Cnt_rst, PWM1_Cnt_INC, PWM2_Cnt_rst, PWM2_Cnt_INC, PWM3_Cnt_rst,
PWM3_Cnt_INC, PWM4_Cnt_rst, PWM4_Cnt_INC : STD_LOGIC:= '0';
signal PWM1_Cnt_Out, PWM2_Cnt_Out, PWM3_Cnt_Out, PWM4_Cnt_Out: STD_LOGIC_VECTOR(15
downto 0):= (others=>'0');

```

```

begin

```

```

    when S1 =>

```

```

        if(PWM4_Cnt_Out < PWM4_DC) then
            NS_PWM4 <= S1;
        else
            NS_PWM4 <= S2;
        end if;
        PWM4_Cnt_INC <= '1';
        LED4_Out <= '1';

    when S2 =>
        if(PWM4_Cnt_Out < PWM_PRD) then
            NS_PWM4 <= S2;
        else
            NS_PWM4 <= S0;
        end if;
        PWM4_Cnt_INC <= '1';

    when others =>
        NS_PWM4 <= S0;

    end case;
end process;
----END Next State Logic for PWM4 ASM
----State Sync
sync_States: process
begin
    wait until clk'event and clk = '1';
    if (rst = '0') then
        CS_Bus <= S0;
    else
        CS_Bus <= NS_Bus;
    end if;
end process;
----End State Sync

----State Sync for PWMs
sync_PWM: process
begin
    wait until clk'event and clk = '1';
    if(rst = '0' or PWM_En = X"0000") then
        CS_PWM1 <= S0;
        CS_PWM2 <= S0;

```



```
    CS_PWM3 <= S0;
    CS_PWM4 <= S0;
else
    CS_PWM1 <= NS_PWM1;
    CS_PWM2 <= NS_PWM2;
    CS_PWM3 <= NS_PWM3;
    CS_PWM4 <= NS_PWM4;
end if;
end process;
----End State Sync

end Behavioral;
```